VALORA PLATFORM ARCHITECTURE

Sandbox Architecture & Capability Assessment

A Technical Review of Production Hardening, Multi-Tenancy & Operational Maturity for Group8Project

1. Executive Summary & Context

Group8Project established the foundational domain architecture for the VALORA platform—defining core models across property management, complaints, work orders, and resident interactions. As the system prepared for pilot deployment and multi-community operation, an advanced iteration (termed **Sandbox**) was developed to resolve operational edge cases, enforce multi-tenant boundaries, integrate automated AI triage, and implement comprehensive automated testing.

This review details the operational and technical enhancements introduced in Sandbox. Rather than replacing the team's work, Sandbox builds upon the foundation of Group8Project, elevating it into a hardened, pilot-ready platform.

> ***Key Takeaway:*** *Sandbox delivers 712 passing backend tests, 13 additive database migration layers, complete multi-tenant estate scoping, an AI operational assist layer, and strict data integrity constraints. Integrating Sandbox equips the platform with an enterprise-grade backend while preserving complete alignment with Group8's vision.*

2. Architectural Enhancements Matrix

Capability Area	Group8 Initial Baseline	Sandbox Production Implementation
Tenancy & Isolation	Single flat repository model; records shared global space without estate tenancy scoping.	Full **Estate Scoping** (Estate, Memberships, Join Codes). Strict row-level tenant boundary validation with resolve_access().
Living Score Engine	Prototype 3-component formula calculated on-demand per property without persistence.	Authoritative **5-Component Living Score v1.0.0** (Resolution 30%, Frequency 25%, Security 20%, Satisfaction 15%, Infrastructure 10%) with rolling 90-day snapshots.
AI Operational Assist	Conceptual AI training data templates without live backend orchestration.	Production **AI Assist Layer** (Gemini / OpenAI / Fallback) with PII redaction, complaint triage suggestions, and AI Health Brief generation.
Security & Rate Limiting	Standard authentication; login attempts tracked via draft unlinked modules.	Account lockout protection (5 failed attempts / 15-min window), dummy bcrypt timing defense, and standardized error envelopes.
Security Incidents	Not represented in data model or APIs.	Dedicated **Security Incidents Engine** with severity classification, status workflow, and direct integration into Living Score calculations.
Public Directory & Media	Static frontend listings without backend image processing pipeline.	Secure public showcase endpoints, image resizing & automated thumbnail generation, and public estate publishing toggles.
Database Migrations	2 initial Alembic revisions.	**13 additive Alembic revisions** enforcing RESTRICT foreign keys, partial indexes, and data consistency.
Test Verification	Modular scripts and manual verification workflows.	**712 automated pytest tests** covering unit, integration, security policies, and API contracts.

3. Technical Deep Dive: Key Pillars Addressed in Sandbox

A. Multi-Tenant Estate Architecture & Row-Level Authorization

Commercial deployment requires strict data boundaries between independent residential communities. Sandbox implements explicit multi-tenancy where users belong to one or more communities via EstateMembership. Every operational entity—complaints, work orders, announcements, and living score calculations—is scoped to an estate_id. Residents join their community through self-service 6-character alphanumeric join codes (managed and rotatable by HOA admins). All data access passes through EstateAuthorizationService.resolve_access(), preventing cross-community data leakage.

B. Authoritative 5-Component Living Score Engine (v1.0.0)

Sandbox formalizes the Living Score algorithm based on architectural consensus (ADR-005). The engine runs on rolling 90-day activity snapshots across five weighted pillars:

- **Issue Resolution Rate (30%):** Percentage and speed of resolved resident maintenance tickets.
- **Complaint Frequency (25%):** Ticket volume normalized per residential unit.
- **Security Incidents (20%):** Volume and severity weighting of recorded community safety events.
- **Resident Satisfaction (15%):** Verified resident star ratings collected upon work order completion.
- **Infrastructure Maintenance (10%):** Proportion of proactive preventative tasks versus emergency repairs.

Results are archived in immutable LivingScoreSnapshot records, providing historical auditability for leadership and prospective buyers.

C. Production AI Operational Assist Layer

Sandbox implements an enterprise AI service abstraction supporting Google Gemini, OpenAI-compatible APIs, and deterministic fallbacks. Outbound prompts undergo automated PII redaction to strip resident names, unit numbers, phone numbers, and email addresses. The system delivers automated complaint categorization & trade matching with an HOA Admin approval workflow (Accept, Override, or Dismiss), and compiles executive operational Health Briefs summarizing critical estate trends.

D. Database Hardening & Security Defenses

Sandbox incorporates 13 additive Alembic migrations that enforce PostgreSQL data integrity constraints: RESTRICT foreign key behaviors prevent accidental cascading deletes, and partial unique indexes prevent duplicate active work orders. Authentication is protected by automated IP-level lockout (5 failed attempts within 15 minutes) and dummy bcrypt timing attack defenses.

4. Recommended Implementation & Transition Strategy

Adopting the Sandbox codebase does not replace the team's architectural intent—it realizes it in a production-ready, verified runtime. Because Sandbox shares the foundational domain entities designed by Group8, migration is additive and safe:

- **Backend Transition:** Replace the nested Group8Project-main/backend directory with Sandbox's clean root backend/. Run alembic upgrade head to apply all 13 migrations.
- **Verification:** Execute the automated test suite (pytest) to confirm all 712 test cases pass cleanly in the environment.
- **Frontend Connectivity:** Follow the companion *Frontend Component Implementation Guide* to connect user interfaces to the newly active endpoints.